

Série d'exercices C

Listes : opération et motifs récurrents de calcul

- On souhaite créer une `Map[Int, String]` à partir d'un `Vector[String]` et d'une `List[Int]`, où les clés sont les éléments de la liste et les valeurs les éléments du vecteur (en se basant sur la collection la plus petite).
 - Ecrivez une première fonction récursive pour combiner les clés et les valeurs. Cette dernière prend le vecteur et la liste en paramètre et retourne une `List[(Int, String)]`.
 - Ecrivez une seconde fonction qui prend le retour de la première fonction en paramètre et qui retourne la `Map[Int, String]`.

- Implémenter l'opération `init` avec le *pattern matching*.

Rappel : `init` retourne une liste contenant tous les éléments sauf le dernier. Une exception est lancée si la liste est vide.

```
def init[T](xs: List[T]): List[T] = xs match
  case List() => throw new Exception("init of empty list")
  case List(x) => ???
  case y :: ys => ???
```

- Trouver l'avant dernier élément d'une liste.

Exemple : `penultimate(List(1, 1, 2, 3, 5, 8))` `//> res0: Int = 5`

```
def penultimate[T](xs: List[T]) = ???
```

- Détecter si une liste est un palindrome.

Exemple : `isPalindrome(List(1, 2, 3, 2, 1))` `//> res0: Boolean = true`

```
def isPalindrome[T](xs: List[T]) = ???
```

- Enlever le $n^{\text{ième}}$ élément d'une liste `xs`. Si n est hors des limites de la liste retourner tous simplement `xs`.

```
def removeAt[T](n: Int, xs: List[T]) = ???
```

Exemple d'utilisation :

```
removeAt(1, List('a', 'b', 'c', 'd'))
//> res1: List[Char] = List(a, c, d)
```

6. Ecrire une fonction `pack`, qui prend les éléments dupliqués consécutifs dans une liste et les regroupe en plusieurs sous-listes.

Par exemple, `pack(List("a", "a", "a", "b", "c", "c", "a"))` doit retourner :

```
List( List("a", "a", "a"),  
      List("b"),  
      List("c", "c"),  
      List("a"))
```

Vous pouvez compléter l'implémentation suivante :

```
def pack[T](xs: List[T]): List[List[T]] = xs match  
  case Nil      => Nil  
  case y :: ys => ???
```

7. En utilisant `pack`, écrire une fonction `encode` qui produit le "run length encoding" (codage par plage) d'une liste.

L'idée est d'encoder les n dupliqués consécutifs d'un élément x comme un paire (x, n) .

Par exemple `encode(List("a", "a", "a", "b", "c", "c", "a"))` doit retourner :

```
List(("a", 3), ("b", 1), ("c", 2), ("a", 1))
```

8. Implémenter une fonction récursive qui sépare une liste donnée en deux sous-listes dont la première est de taille n . Vous ne devez pas utiliser les méthodes `take` et `drop`. Compléter uniquement les parties (a), (b) et (c).

```
def splitRecursive[A](n: Int, ls: List[A]): (List[A], List[A]) =  
  ls match  
    case Nil      => ??? // a  
    case list if n <= 0 => ??? // b  
    case h :: tail => ??? // c
```

9. Ecrire la méthode `decode` qui permet de décoder une liste encodée (l'inverse de l'exercice d'encodage de la question 7) seulement en utilisant le pattern matching.

Par exemple : `decode(List(("a", 3), ("b", 1), ("c", 2), ("a", 1)))` doit retourner :

```
List("a", "a", "a", "b", "c", "c", "a")
```

10. Implémenter la fonction `takeWhileStrictlyIncreasing` qui, étant donné une liste d'entiers, 1, retourne la plus grande suite des éléments qui sont strictement croissante.

Quelques exemples d'application :

```
takeWhileStrictlyIncreasing(List(1, 8, 9, 9, 10, 2, 3))  
// List(1, 8, 9)  
takeWhileStrictlyIncreasing(List(9, 10, 1, 2, 3, 11, 12, 13))  
// List(9, 10)  
takeWhileStrictlyIncreasing(List())  
// List()  
  
def takeWhileStrictlyIncreasing(list: List[Int]): List[Int] = ???
```